Recitation 5 – RFC, DNS, and TLS

Daniel Karalnik and Liam Tal

Computer Networks

Table of Contents

1. DNS
2. Recap and Questions
3. Request For Comments (RFC)
4. TLS

Today

- DNS (Domain Name System) - continued
- True/False Exam Questions
- TLS (Transport Layer Security)

DNS Security - Cache Poisoning

Historical vulnerabilities and mitigations:

1997: Bailiwick Checking

Problem: Attackers could inject additional records from different domains in the response
Solution (Bailiwick Checking): Only accept resource records that are within the authority ("bailiwick") of the queried domain
Example:

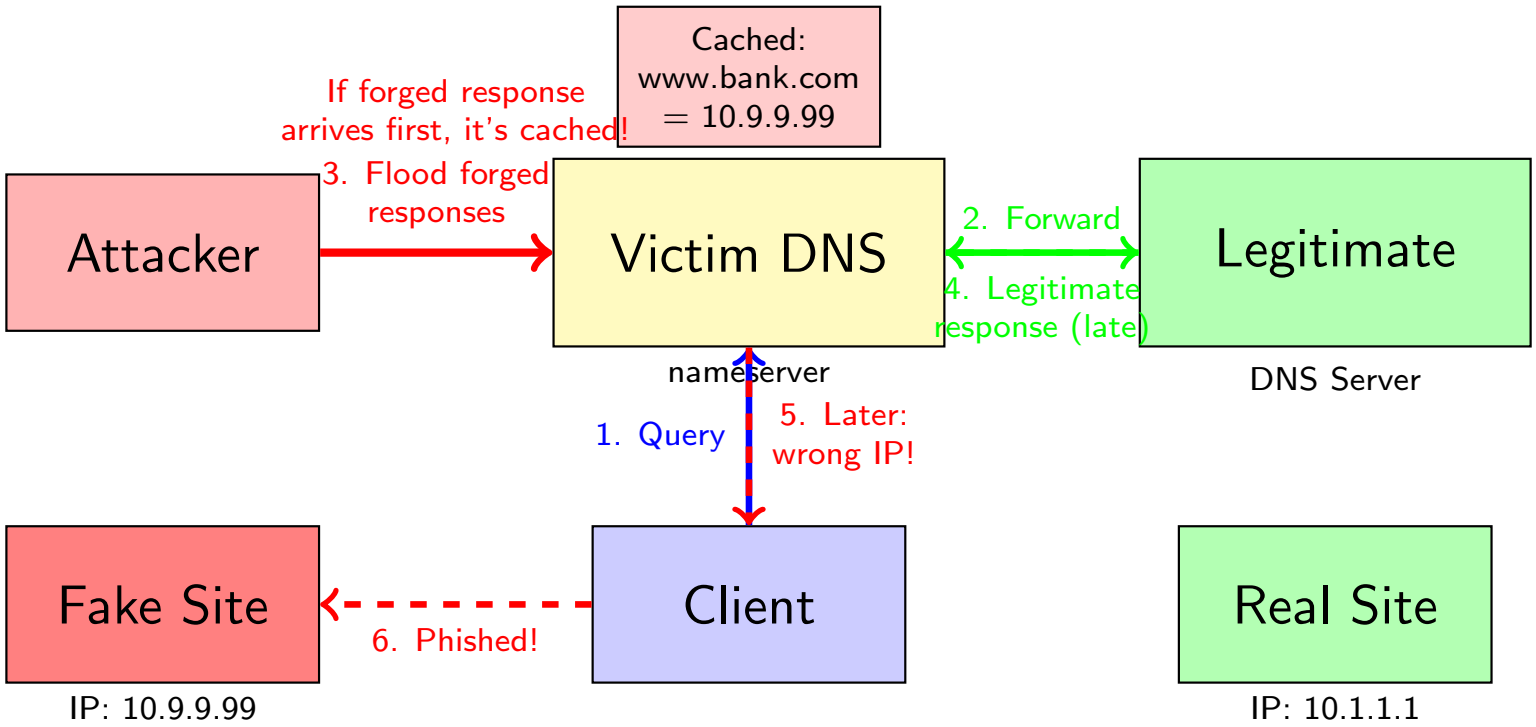
- Query for `www.bank.com`
- Accept answers about `*.bank.com` (within bailiwick)
- Reject answers about `www.attacker.com` (outside bailiwick)

DNS Security - Cache Poisoning

DNS Cache Poisoning Attack

Problem: Transaction IDs were predictable (often incremental/sequential)
Attack: Attacker sends many forged responses with guessed IDs, racing to respond before the legitimate DNS server
Mitigation: Randomize transaction IDs (16 bits), use source port randomization (+16 bits), implement DNSSEC

DNS Cache Poisoning - Attack Diagram



Reference: Understanding Kaminsky's DNS Bug

DNS Security - Modern Mitigations

Randomization Techniques

Transaction ID + Source Port Randomization:

- 16-bit random transaction ID (65,536 possibilities)
- 16-bit random source port (additional 65,536 possibilities)
- Combined: $2^{32} = 4.3$ billion combinations
- Makes cache poisoning attacks impractical within the small time window

DNS Security - Modern Mitigations

DNSSEC

- Cryptographic signatures
- Chain of trust from root
- Prevents cache poisoning
- Authenticates DNS data

Best Practices

- Use trusted DNS servers
- Enable DNSSEC validation
- Keep DNS software updated
- Monitor for anomalies

DNSSEC - DNS Security Extensions

What is DNSSEC?

DNSSEC adds cryptographic authentication to DNS responses, ensuring that the data you receive actually came from the authoritative source and hasn't been tampered with.

Digital Signatures

- Each DNS record is digitally signed
- Uses public/private key cryptography
- Signature proves authenticity

Chain of Trust

- Root DNS signs TLD keys
- TLD DNS signs domain keys
- Domain signs its records
- Validates entire path

DNSSEC - Benefits and Limitations

Benefits of DNSSEC

Key Security Improvements:

- **Prevents DNS cache poisoning:** Forged responses are detected and rejected
- **Authenticates DNS data:** Ensures responses come from authoritative source
- **Integrity verification:** Data hasn't been modified in transit
- **Building block for security:** Enables DANE, TLSA, and other security protocols

DNSSEC - Benefits and Limitations

Limitations of DNSSEC

Current Challenges:

- **No confidentiality:** Queries/responses still visible (combine with DNS over HTTPs/DNS over TLS for privacy)
- **Complex deployment:** Key management, rotation, and zone signing complexity
- **Larger responses:** Signatures add overhead (can exceed 512 bytes)
- **Not universally adopted:** Many domains still don't use DNSSEC

DNS Privacy - Modern Protocols

Traditional DNS Problem:

- DNS queries sent in **plaintext** over UDP port 53
- ISP, network operators can see all your DNS queries
- Privacy concern: DNS reveals all websites you visit

Modern Solutions:

- **DNS over TLS (DoT) - RFC 7858:**
  - Encrypts DNS queries using TLS on port 853
  - Dedicated port makes it easy to identify/block
- **DNS over HTTPS (DoH) - RFC 8484:**
  - Encrypts DNS queries over HTTPS (port 443)
  - Harder to distinguish from regular web traffic
  - Supported by Firefox, Chrome, Edge

Trade-off: Better privacy, but centralization concerns (queries go to few large providers)

Question 1 - DNS Queries

True or False:
Iterative queries place less load on a root DNS server than recursive queries.

Question 1 - DNS Queries

True or False:
Iterative queries place less load on a root DNS server than recursive queries.

Answer: True

In iterative queries, the root server only provides a referral to the TLD server. In recursive queries, the root server would need to do all the work of resolving the query itself, placing more load on the root server.

Question 2 - Circuit Switching

True or False:
FDM and TDM are two approaches used in virtual circuit networks.

Question 2 - Circuit Switching

True or False:
FDM and TDM are two approaches used in virtual circuit networks.

Answer: False

FDM (Frequency Division Multiplexing) and TDM (Time Division Multiplexing) are two approaches used in **Circuit-Switched Networks**, not virtual circuit networks.
Virtual circuit networks are a form of packet switching.

Question 3 - Virtual Circuits

Scenario: Users share a single 1 Mbps link, each user requires 0.5 Mbps when transmitting, but each user transmits only 10% of the time.
True or False:
When virtual circuit network is used, up to 2 users can be supported.

Question 3 - Virtual Circuits

Scenario: Users share a single 1 Mbps link, each user requires 0.5 Mbps when transmitting, but each user transmits only 10% of the time.
True or False:
When virtual circuit network is used, up to 2 users can be supported.

Answer: False

Virtual circuit network is a specialized form of packet switching, not circuit switching. With packet switching and 10% utilization, many more than 2 users can be supported. Two users would only use 10% of the link capacity on average (2 users $\times$ 0.5 Mbps $\times$ 0.1 $=$ 0.1 Mbps).

Question 4 - HTTP Connections

Scenario: A web page (HTML) has **nine** embedded GIF images. The **10 objects** (page + 9 images) reside at **two different servers**: the page plus 8 images on the **first server**, and a **single image** on the **second server**.

True or False:
The response time for **persistent connection with pipelining and parallel connections** will be **faster** than for **non-persistent connection with parallel connections**. (Assume no bound on parallel connections.)

Approach 1: Persistent with Pipelining - Timeline

Total: 4 RTTs

- RTT 1: TCP setup to Server A
- RTT 2: Get HTML from Server A
- RTT 3: Image #1 request + TCP setup for others (parallel)
- RTT 4: Receive all images

Approach 2: Non-Persistent with Parallel - Timeline

Total: 4 RTTs

- RTT 1: TCP setup to Server A
- RTT 2: Get HTML from Server A, close connection
- RTT 3: TCP setup for all 9 images (parallel)
- RTT 4: Get all images, close all connections

Answer: False - They Take the Same Time!

Answer: FALSE

The response time for persistent connection with pipelining will **NOT** be faster than non-persistent with parallel connections.
Both require exactly 4 RTTs.

Question 5 - HTTP Pipelining

True or False:

Non-persistent HTTP connections can use pipelining.

Question 5 - HTTP Pipelining

True or False:
Non-persistent HTTP connections can use pipelining.

Answer: False

Non-persistent HTTP connections only send and receive a single request/response pair. After the response, the connection is closed. Pipelining requires sending and receiving multiple requests/responses over the same connection, which requires a persistent connection.

Request For Comments (RFC)

What is an RFC?

Definition

RFCs are technical documents that describe Internet standards, protocols, and specifications. They are maintained by the Internet Engineering Task Force (IETF).

RFC Categories:

- **Standards Track:** Official Internet standards
- **Informational:** General information about topics
- **Experimental:** Protocols still under development
- **Best Current Practice:** Guidelines for Internet operations

Echo Protocol - RFC 862

Exercise: Review the Echo Protocol RFC (862)
https://tools.ietf.org/html/rfc862

> This is one of the shortest RFCs! The Echo Protocol simply sends back any data it receives - useful for testing network connections.

Important RFCs for Networking:

- **RFC 791:** Internet Protocol (IP)
- **RFC 793:** Transmission Control Protocol (TCP)
- **RFC 768:** User Datagram Protocol (UDP)
- **RFC 1035:** Domain Name System (DNS)
- **RFC 2616:** HTTP/1.1 (superseded by RFC 7230-7235)
- **RFC 5321:** Simple Mail Transfer Protocol (SMTP)

Why TLS?

Three main security goals:

Confidentiality

Protecting information from disclosure to unauthorized parties

Integrity

Protecting information from being modified by unauthorized parties

Authentication (optional)

Ensuring you really communicate with the partner you intend to

TLS - Transport Layer Security

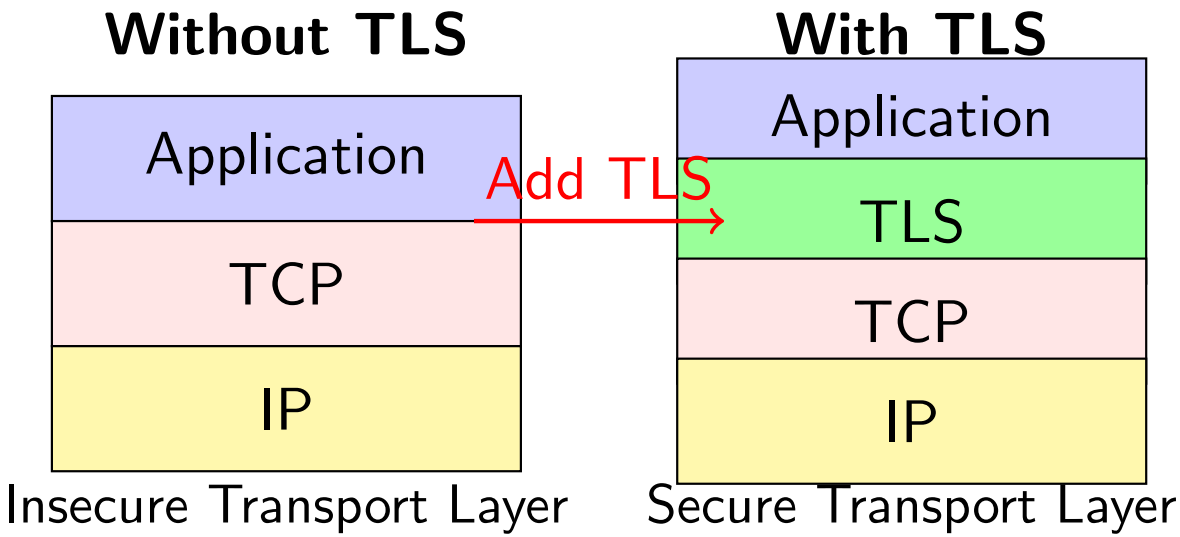
What is TLS?

Transport Layer Security Protocol

- De facto standard for Internet security
- Commonly used to protect HTTP traffic (HTTPS)
- Also protects SMTP, POP3, FTP, and many other protocols
- Deployed in nearly every web browser

Note: TLS is the successor to SSL (Secure Sockets Layer). The terms are often used interchangeably, but TLS is the modern standard.

TLS - Layer Position



TLS sits between the application layer and TCP, providing security without modifying the application protocol itself.

TLS - Cryptography Basics

Symmetric Encryption:

- Same key for encryption and decryption
- Fast and efficient
- Problem: How to securely share the key?
- Example: AES, One-Time Pad

Asymmetric Encryption (Public Key):

- Two keys: public key (encrypt) and private key (decrypt)
- Public key can be shared openly
- Only the private key can decrypt
- Slower than symmetric encryption
- Example: RSA

TLS uses both: Asymmetric encryption to establish a shared secret, then symmetric encryption for the actual data transfer.

TLS - Symmetric Encryption Example

Example: One-Time Pad (OTP)

Given a message M: 1 1 0 1 0 ... 1 1 0 1 0 1
And a symmetric (shared) key K: 0 1 0 0 0 ... 1 0 0 0 1 1

$\otimes$ (XOR operation)

We get ciphered message C: 1 0 0 1 0 ... 0 1 0 1 1 0

- XOR operation: $1 \oplus 0 = 1$, $0 \oplus 0 = 0$, $1 \oplus 1 = 0$, $0 \oplus 1 = 1$
- Can decrypt only using the correct key
- Apply XOR with same key to recover original message

TLS - Certificate Chain

How do we trust the public key?

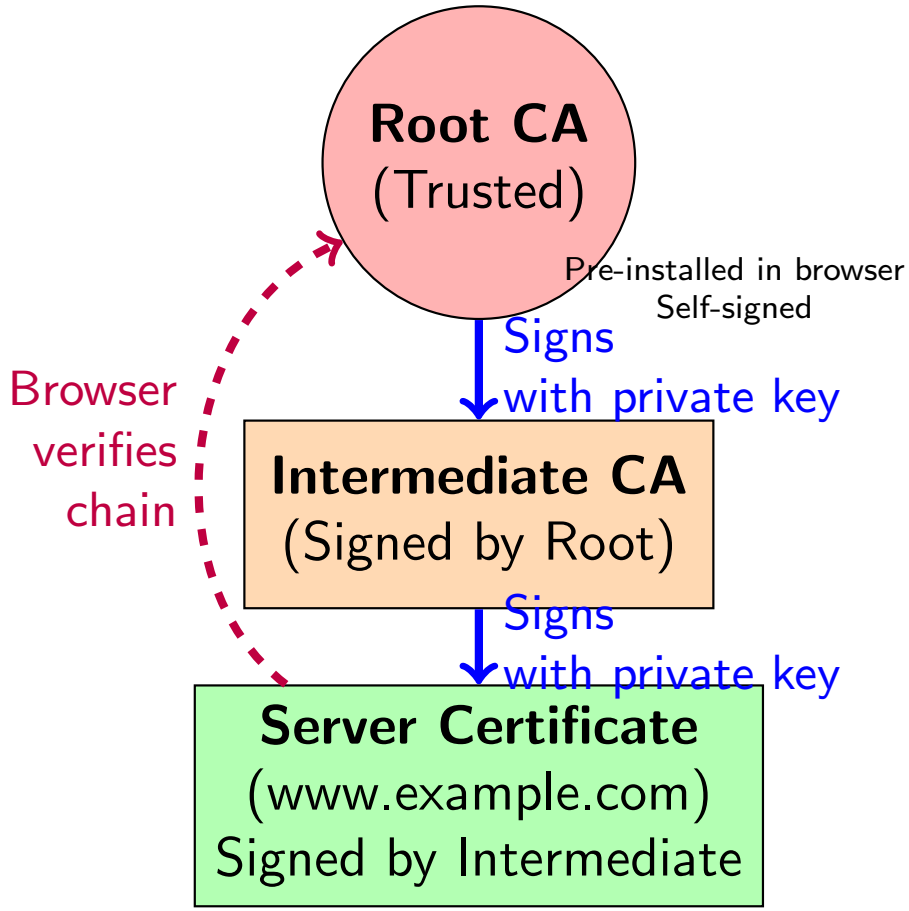
Certificate Authority (CA)

A trusted third party that signs certificates to verify identity:

- Server has a certificate signed by a CA
- Certificate contains server's public key
- Your browser trusts specific CAs
- Browser verifies the certificate chain

Important: This prevents Man-in-the-Middle attacks by verifying the server's identity.

Certificate Chain - Visual



Each certificate is signed by the one above it. Browser verifies signatures up to a trusted root.

TLS Versions - Evolution

History of SSL/TLS:

- **SSL 2.0 (1995):** First widely deployed version - DEPRECATED
- **SSL 3.0 (1996):** Major redesign - DEPRECATED (POODLE attack)
- **TLS 1.0 (1999):** Renamed to TLS - Being phased out
- **TLS 1.1 (2006):** Protection against CBC attacks - Being phased out
- **TLS 1.2 (2008):** Modern cipher suites - Still widely used
- **TLS 1.3 (2018):** Faster handshake, removed weak crypto - Current standard

Modern practice: Use TLS 1.2 or 1.3 only. Older versions have known vulnerabilities.

TLS - Handshake Protocol

TLS consists of two main protocols:

Handshake Protocol

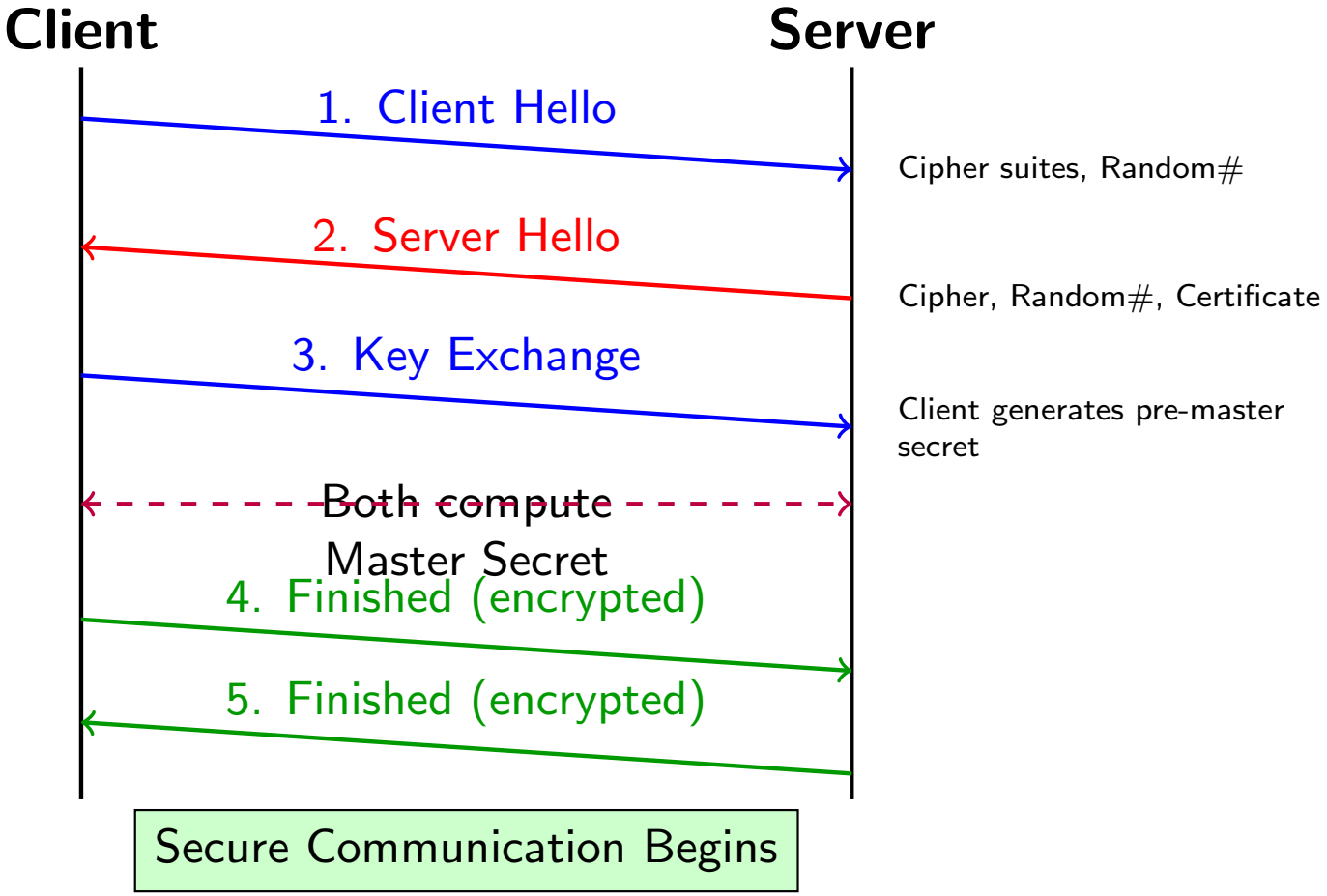
Establishes a shared secret key between client and server:

1. Client Hello (supported cipher suites, random number)
2. Server Hello (chosen cipher, random number, certificate)
3. Key exchange
4. Both sides compute shared secret
5. Handshake complete

Record Protocol

Uses the shared secret to encrypt and authenticate all communication

TLS Handshake - Visual Flow



Key Point: Asymmetric crypto for handshake, symmetric crypto for data transfer, in practice under TLS1.3 can be done in 1 RTT

HTTPS RTT Calculation - Question

Scenario:
A client wants to fetch a simple HTML page from `https://www.example.com`. The page has no embedded objects.

Given:

- RTT between client and server: **100 ms**
- Using TLS 1.2 (requires 2-RTT handshake)
- Persistent connection
- Ignore transmission time and processing time

Question: How long does it take from when the client initiates the connection until it receives the complete HTML page?

HTTPS Connection - What's Needed?

Remember: HTTPS = HTTP over TLS over TCP

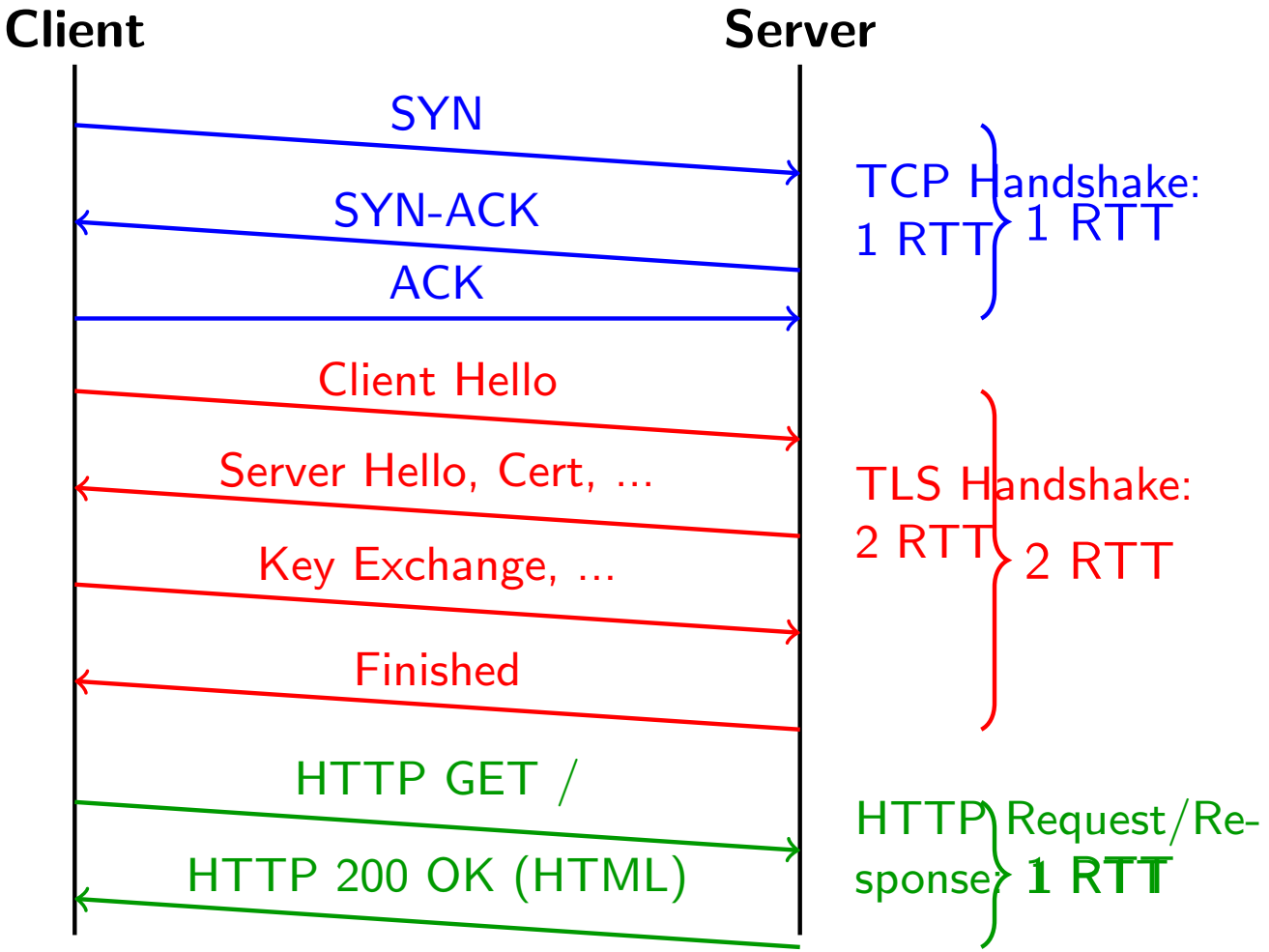
To establish an HTTPS connection and fetch data, we need:

1. **TCP Handshake** (SYN, SYN-ACK, ACK)
2. **TLS Handshake** (establish encryption)
3. **HTTP Request/Response** (GET and receive HTML)

Tip: Count how many round trips each step requires!

Think about your answer, then proceed to the next slide for the detailed solution.

Solution - Visual Timeline



Solution - Calculation

Step-by-step calculation:

1. **TCP Handshake:** 1 RTT = 100 ms
   - Client sends SYN
   - Server responds with SYN-ACK (1 RTT complete)
   - Client sends ACK (can piggyback data, so no extra RTT)
2. **TLS 1.2 Handshake:** 2 RTT = 200 ms
   - Client Hello → Server Hello, Certificate (1 RTT)
   - Key Exchange → Finished (1 RTT)
3. **HTTP Request/Response:** 1 RTT = 100 ms
   - Client sends HTTP GET
   - Server responds with HTML (1 RTT)

Final Answer

$$\textbf{Total Time} = 1 + 2 + 1 = 4 \textbf{ RTT} = 4 \times 100 \textbf{ ms} = 400 \textbf{ ms}$$

Bonus Question

What if the HTML page has 3 embedded images from the same server? How much additional time to fetch all 3 images?

Assume:

- Persistent connection (already established)
- HTTP/1.1 with pipelining
- RTT = 100 ms

Bonus Question

Answer

1 additional RTT = 100 ms

Why? With pipelining on a persistent connection:

- Send all 3 image requests at once (no waiting)
- Receive all 3 responses
- Total: 1 RTT

Grand Total: 400 ms + 100 ms = 500 ms

Comparison: HTTPS vs HTTP

For the same scenario (HTML + 3 images):

HTTP (non-secure)

- TCP: 1 RTT
- TLS: 2 RTT
- HTML: 1 RTT
- Images: 1 RTT

HTTPS (secure)

- TCP: 1 RTT
- TLS: 2 RTT
- HTML: 1 RTT
- Images: 1 RTT

Total: 3 RTT
= 300 ms

Total: 5 RTT
= 500 ms

Security has a cost: HTTPS adds 2 RTT (200 ms) for the TLS handshake, but it's worth it for encryption and authentication!

Key Takeaways

Important points to remember:

1. **HTTPS = HTTP + TLS + TCP**
   - Don't forget to count the TLS handshake!
2. **TLS 1.2 requires 2 RTT** for handshake
   - TLS 1.3 improves this to 1 RTT (or 0-RTT for resumption)
3. **TCP handshake: 1 RTT** (always needed first)
4. **Persistent connections** save time for multiple requests
   - Reuse TCP and TLS connections
5. **Pipelining** allows multiple HTTP requests in 1 RTT